



EXPERIMENT 4

Name : Abhishek Pandey

Subject: Machine Learning

SAP ID: 60019220110

AIM: To implement Logistic Regression

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

1. Load Dataset from URL
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
data = pd.read_csv(url)

# Convert to binary classification
data["target"] = (data["species"] != "setosa").astype(int)

# Use only 2 features for visualization
X = data[["sepal_length", "sepal_width"]].values
y = data["target"].values

# Feature scaling
X = (X - X.mean(axis=0)) / X.std(axis=0)

2. Logistic Regression Class

class LogisticRegression:
    def __init__(self, learning_rate=0.1, n_iters=2000):
        self.lr = learning_rate
        self.n_iters = n_iters

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))
```



```
def fit(self, X, y):
```

```
    n_samples, n_features = X.shape
```

```
    self.weights = np.zeros(n_features)
```

```
    self.bias = 0
```

```
    for _ in range(self.n_iters):
```

```
        linear_model = np.dot(X, self.weights) + self.bias
```

```
        y_pred = self.sigmoid(linear_model)
```

```
        dw = (1 / n_samples) * np.dot(X.T, (y_pred - y))
```

```
        db = (1 / n_samples) * np.sum(y_pred - y)
```

```
        self.weights -= self.lr * dw
```

```
        self.bias -= self.lr * db
```

```
def predict(self, X):
```

```
    linear_model = np.dot(X, self.weights) + self.bias
```

```
    y_pred = self.sigmoid(linear_model)
```

```
    return np.where(y_pred >= 0.5, 1, 0)
```

3. Train Model

```
model = LogisticRegression(learning_rate=0.1, n_iters=3000)
```

```
model.fit(X, y)
```

```
predictions = model.predict(X)
```

```
accuracy = np.mean(predictions == y)
```

```
print("Accuracy:", accuracy)
```

4. Plot Data + Decision Boundary

```
plt.figure(figsize=(8,6))
```



```
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='bwr', alpha=0.7)
```

```
x_values = np.array([X[:, 0].min(), X[:, 0].max()])
```

```
y_values = -(model.weights[0] * x_values + model.bias) / model.weights[1]
```

```
plt.plot(x_values, y_values, color='black')
```

```
plt.xlabel("Sepal Length (scaled)")
```

```
plt.ylabel("Sepal Width (scaled)")
```

```
plt.title("Logistic Regression on Iris Dataset")
```

```
plt.show()
```

OUTPUT:



Conclusion: In this experiment, Logistic Regression was successfully implemented from scratch using Python and applied to the Iris dataset for binary classification. The model was trained using gradient descent and achieved good accuracy, demonstrating its effectiveness in solving classification problems. The decision boundary plotted on the graph clearly separates the two classes, showing that Logistic Regression is a simple yet powerful supervised learning algorithm for linearly separable data.